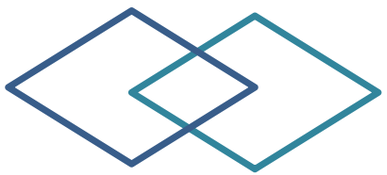


第8章

DMA



本章知识与能力要求

- 理解和掌握DMA的基本概念、适用场合；
- 理解STM32的DMA结构及工作原理；
- 熟悉STM32标准外设库和HAL库中有关DMA的库函数；
- 掌握基于STM32标准外设库开发DMA应用程序的方法；
- 掌握基于STM32CubeMX进行DMA开发的方法。

第8章 DMA

8.1 DMA基础理论知识

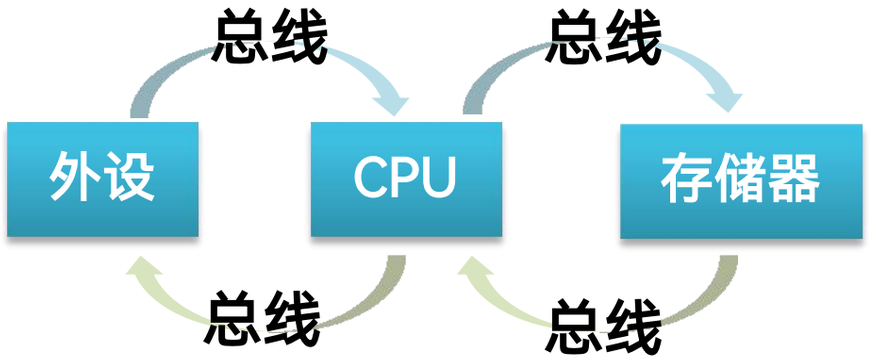
8.2 STM32的DMA模块

8.3 DMA模块的标准外设库接口函数及应用

8.4 DMA模块的HAL库接口函数及应用

8.1 DMA基础理论知识

一般的数据处理流程



在外设与存储器传输数据过程中，CPU全程参与

不需要处理数据时

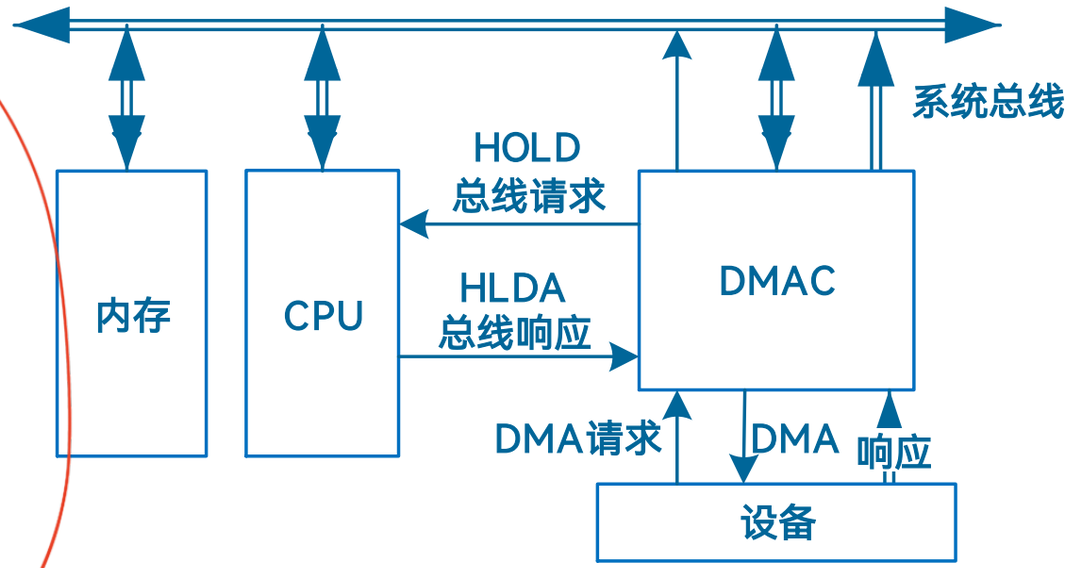
由于外设和存储器速度较低

导致CPU大量的时间和资源被浪费，造成CPU效率低下。

8.1 DMA基础理论知识

DMA传输方式是利用DMA控制器直接控制总线，在外设与存储器之间建立一条直接通道，不需要CPU的中转。

涉及到总线控制权转移问题，即DMA传输前，CPU需要把总线控制权交给DMA控制器，DMA传输结束后，DMA控制器再把总线控制权交还给CPU。



8.1 DMA基础理论知识

DMA传输过程

DMA请求

- 对DMA进行初始化，I/O设备准备好后，向DMA控制器发出DMA请求。

DMA响应

- DMA控制器对DMA请求进行优先级判别，向CPU发出总线请求信号。CPU响应总线请求，当CPU执行完当前总线周期就将总线控制权交给DMA控制器，完成DMA响应。

DMA传输

- DMA控制器获得总线控制权后，外设经硬件信号请求DMA传输或通过软件启动DMA请求，按配置好的相关参数，DMA控制器发出读写命令，开始进行数据传送。

DMA结束

- DMA控制器对传输的数据进行计数，判断数据是否传输完成，当传输完成，DMA控制器将总线控制权交还给CPU，并向I/O设备发出结束信号。

8.2 STM32的DMA模块



8.2.1 DMA内部结构



8.2.2 DMA优先权



8.2.3 DMA中断请求

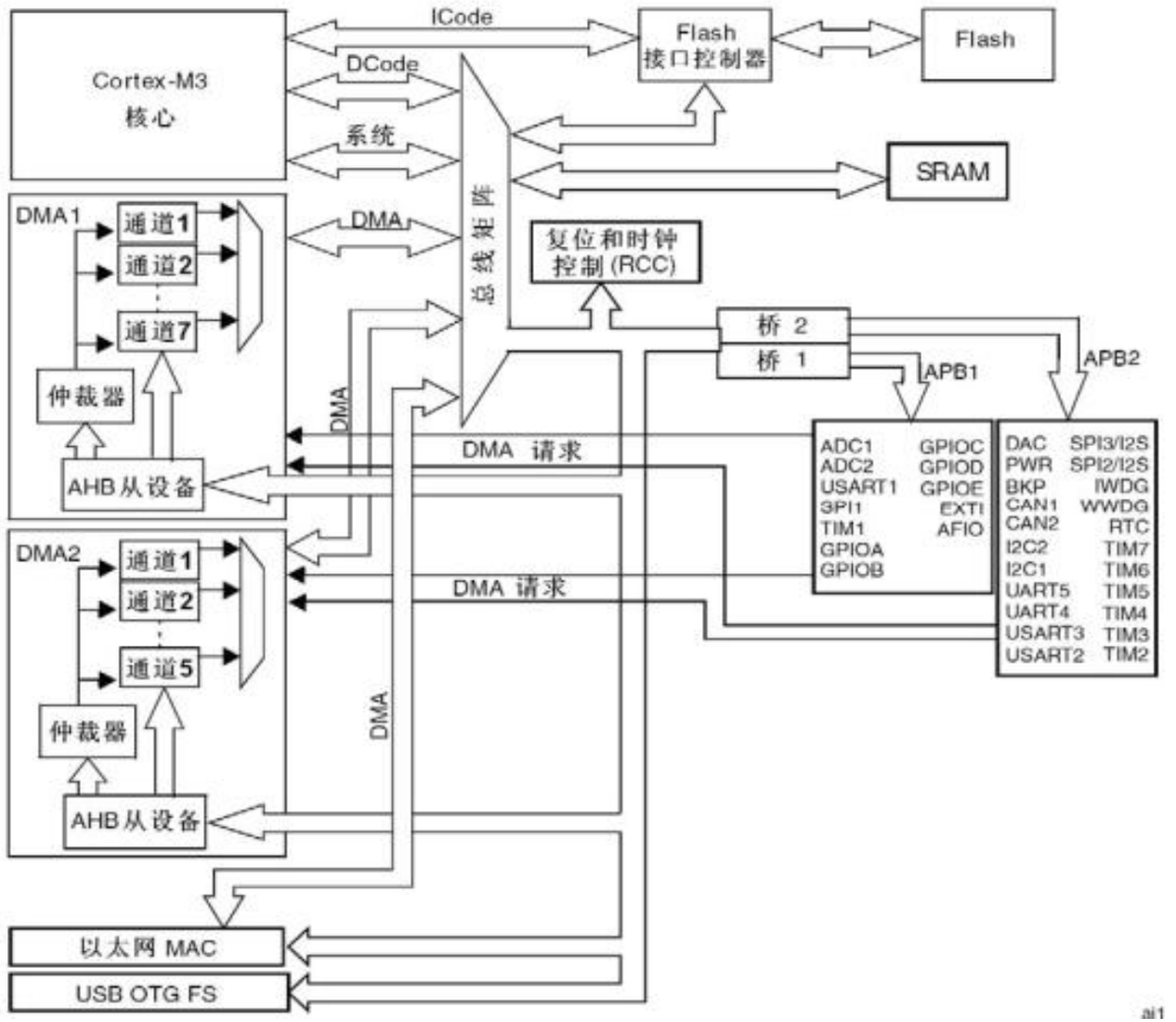
8.2.1 DMA内部结构

STM32最多有2个DMA控制器：

- DMA1有7个通道
- DMA2有5个通道

每一个通道专门用来管理一个或多个外设对存储器访问的请求。

中小容量的STM32F103系列只有DMA1，STM32F103ZET6属于大容量芯片，有DMA1和DMA2两个DMA控制器。



DMA的内部结构框图

8.2.1 DMA内部结构

DMA1各个通道的DMA请求

外设(TIMx(x=1,2,3,4) 、ADC1、SPI1、IICx(x=1,2) 、USARTx(x=1,2,3))产生的DMA1请求，传送到DMA1控制器，同一时刻只能有一个请求有效。

外设	通道1	通道2	通道3	通道4	通道5	通道6	通道7
ADC1	ADC1						
SPI/I2S		SPI1_RX	SPI1_TX	SPI/I2S_RX	SPI/I2S_TX		
USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
IIC				IIC2_TX	IIC2_RX	IIC1_TX	IIC1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP

8.2.1 DMA内部结构

DMA2各个通道的DMA请求

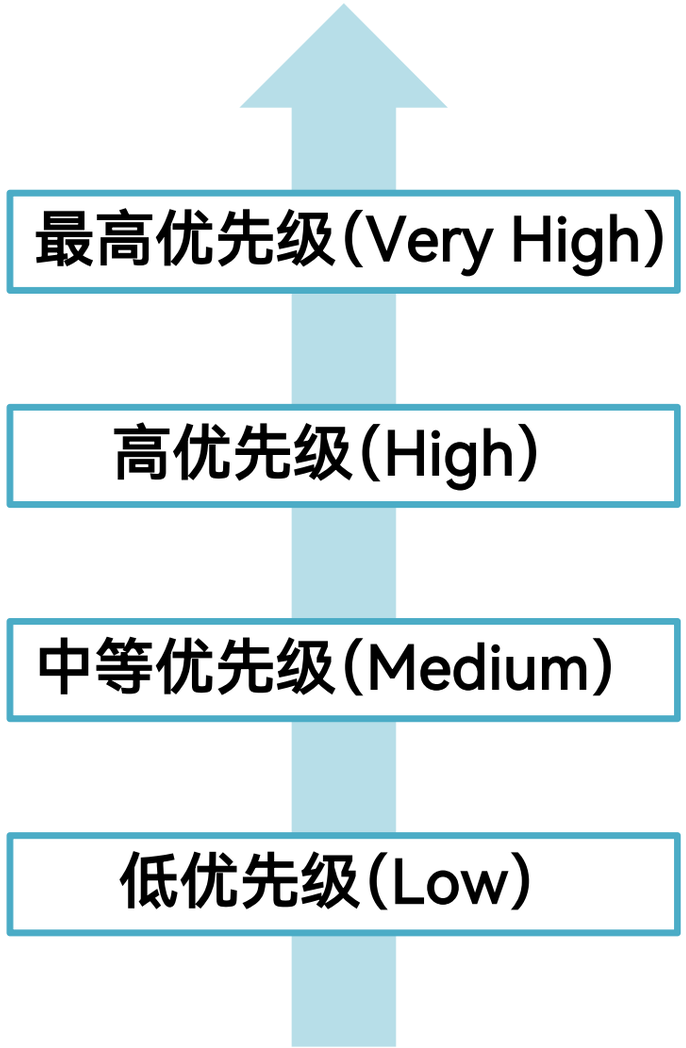
外设(TIMx(x=5,6,7,8) 、ADC3、SPI/I2S3、USART4、DAC通道1、2和SDIO)产生5个通道的DMA2请求传送到DMA2控制器，同一时刻只能有一个请求有效

外设	通道1	通道2	通道3	通道4	通道5
ADC3					ADC3
SPI/I2S3	SPI/I2S3_RX	SPI/I2S3_TX			
USART4			USART4_RX		USART4_TX
SDIO				SDIO	
TIM5	TIM5_CH4 TIM5_TRIG	TIM5_CH3 TIM5_UP		TIM5_CH2	TIM5_CH1
TIM6/ DAC通道1			TIM6_UP/ DMA通道1		
TIM7/ DAC通道2				TIM7_UP/ DMA通道2	
TIM8	TIM8_CH3 TIM8_UP	TIM8_CH4 TIM8_TRIG TIM8_COM	TIM8_CH1		TIM8_CH2

8.2.2 DMA优先权

当有多个DMA请求时，DMA控制器通过内部的仲裁器进行优先权管理。

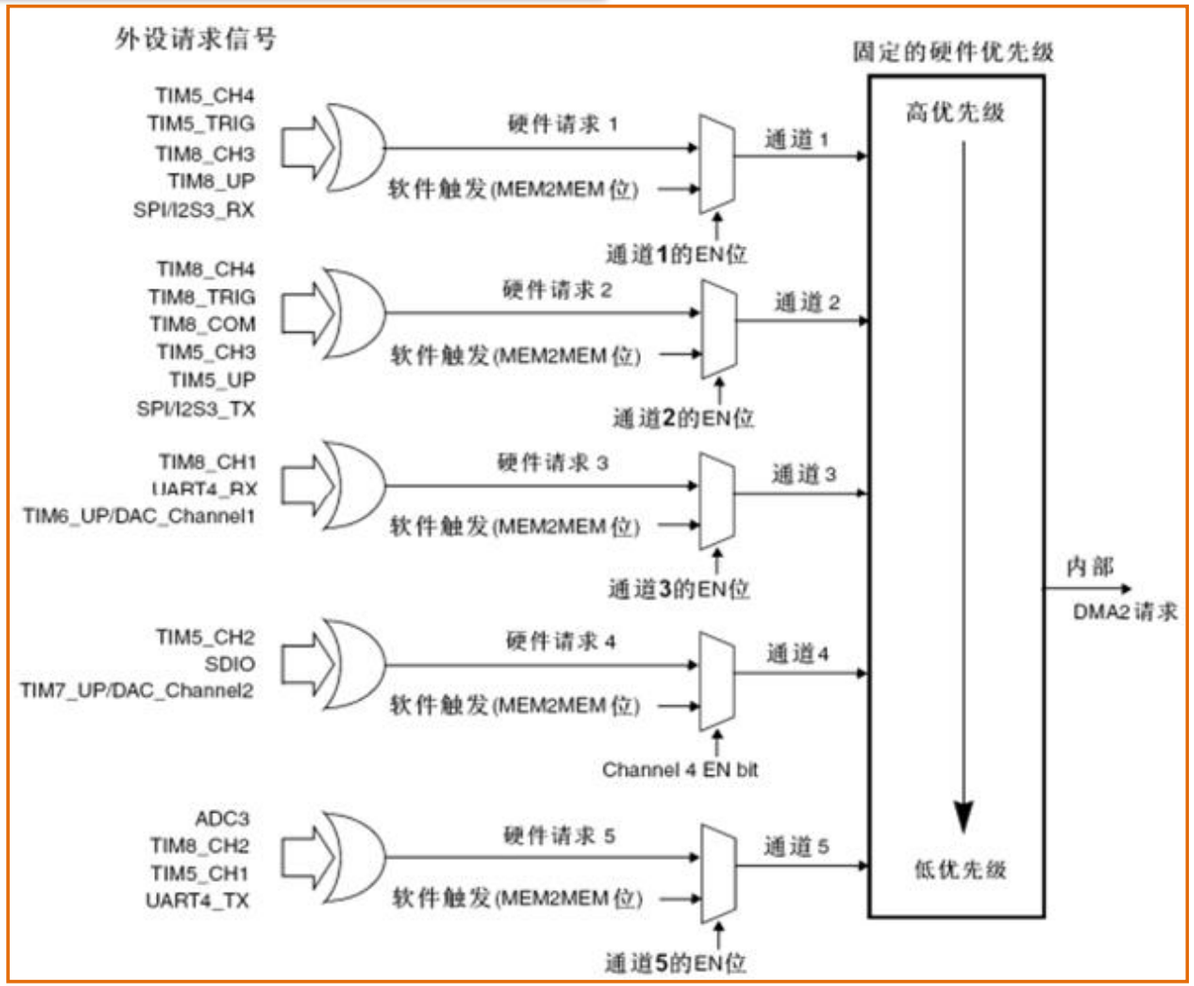
通道的优先权分为4个等级，高优先级的通道优先获得总线响应。



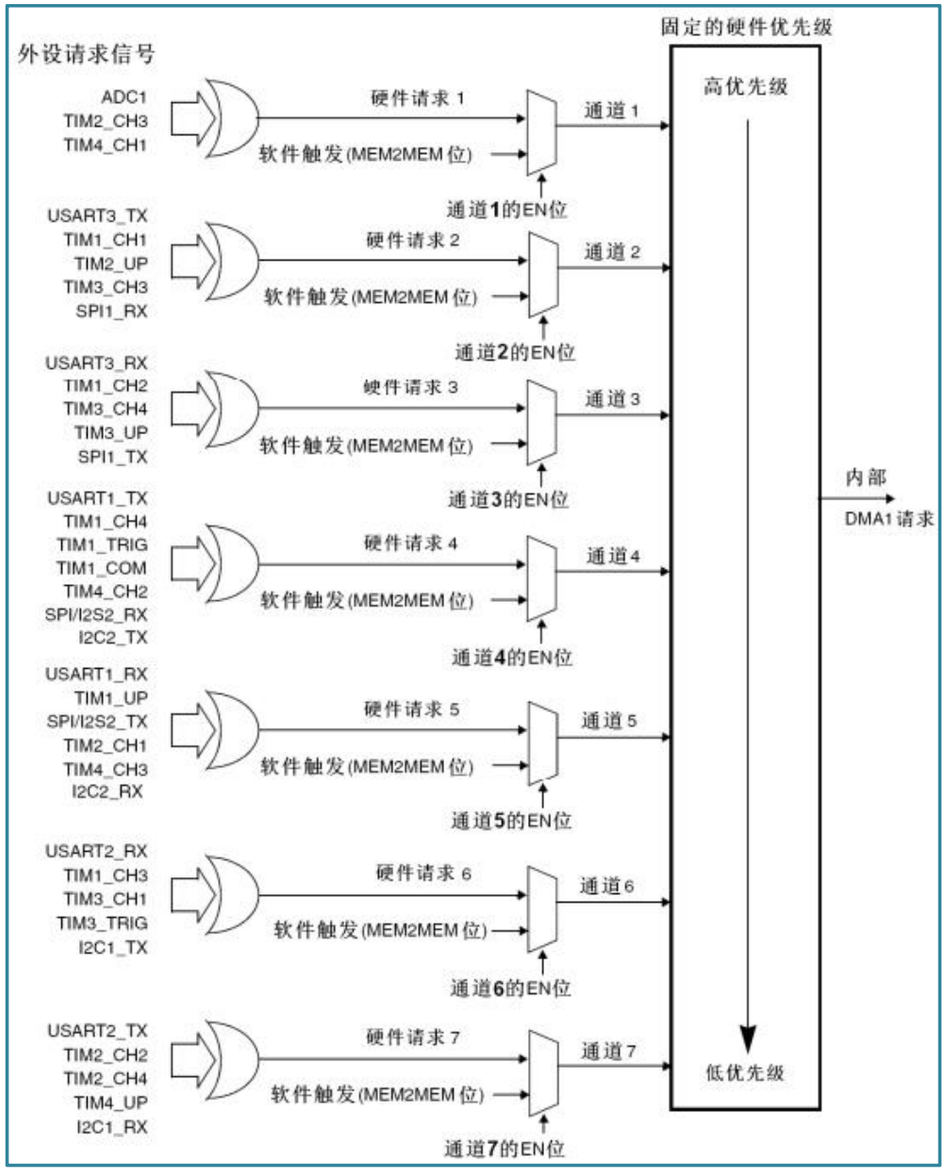
如果2个请求有相同的软件优先级，则较低编号的通道比较高编号的通道有较高的优先权。例如，通道2优先于通道4。

在大容量产品和互联型产品中，DMA1控制器拥有高于DMA2控制器的优先级。

8.2.2 DMA优先权



DMA2各通道的DMA映射及优先权



DMA1各通道的DMA映射及优先权

8.2.3 DMA中断请求

DMA的每个通道都可以在DMA传输过程中触发中断，可通过设置相应寄存器的不同位来打开这些中断。

DMA中断事件主要有：

- HT(Half Transfer，传输一半)
- TC(Transfer Complete，传输完成)
- TE(Transfer Error，传输错误)

分别对应三个中断标志：HTIF、TCIF、TEIF，每个中断标志都有允许控制位。

中断事件标志位	中断使能控制位	中断事件
HTIF	HTIE	传输过半
TCIF	TCIE	传输完成
TEIF	TEIE	传输错误

8.3 DMA模块的标准外设库接口函数及应用

1. 8.3.1 DMA标准外设库接口函数
2. 8.3.2 DMA标准外设库的配置过程
3. 8.3.3 DMA标准外设库应用实例

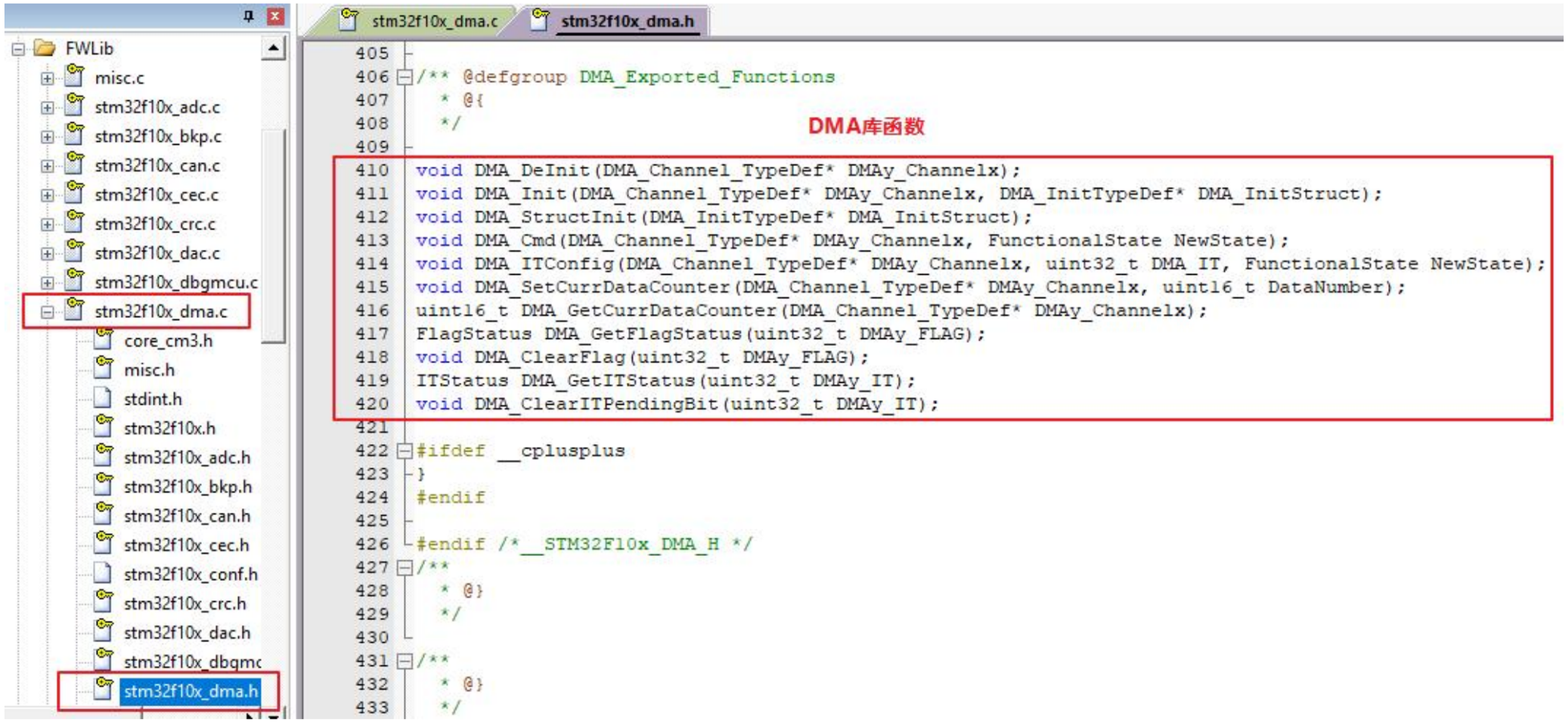
8.3.1 DMA标准外设库接口函数

DMA标准外设库支持11种库函数

函数名称	功能描述
DMA_DeInit	将DMA的通道x寄存器重设为缺省值
DMA_Init	根据DMA_InitStruct中指定的参数初始化DMA的通道x寄存器
DMA_StrucInit	把DMA_InitStruct中的每个参数按缺省值填入
DMA_Cmd	使能或者失能指定的通道x
DMA_ITConfig	使能或者失能指定的通道x中断
DMA_GetCurrDataCounter	返回当前DMA通道x剩余的待传输数据个数
DMA_SetCurrDataCounter	设置DMA通道x进行DMA传输的当前数据个数
DMA_GetFlagStatus	检查指定的DMA通道x标志位设置与否
DMA_ClearFlag	清除DMA通道x待处理标志位
DMA_GetITStatus	检查指定的DMA通道x中断发生与否
DMA_ClearITPendingBit	清除DMA通道x中断待处理标志位

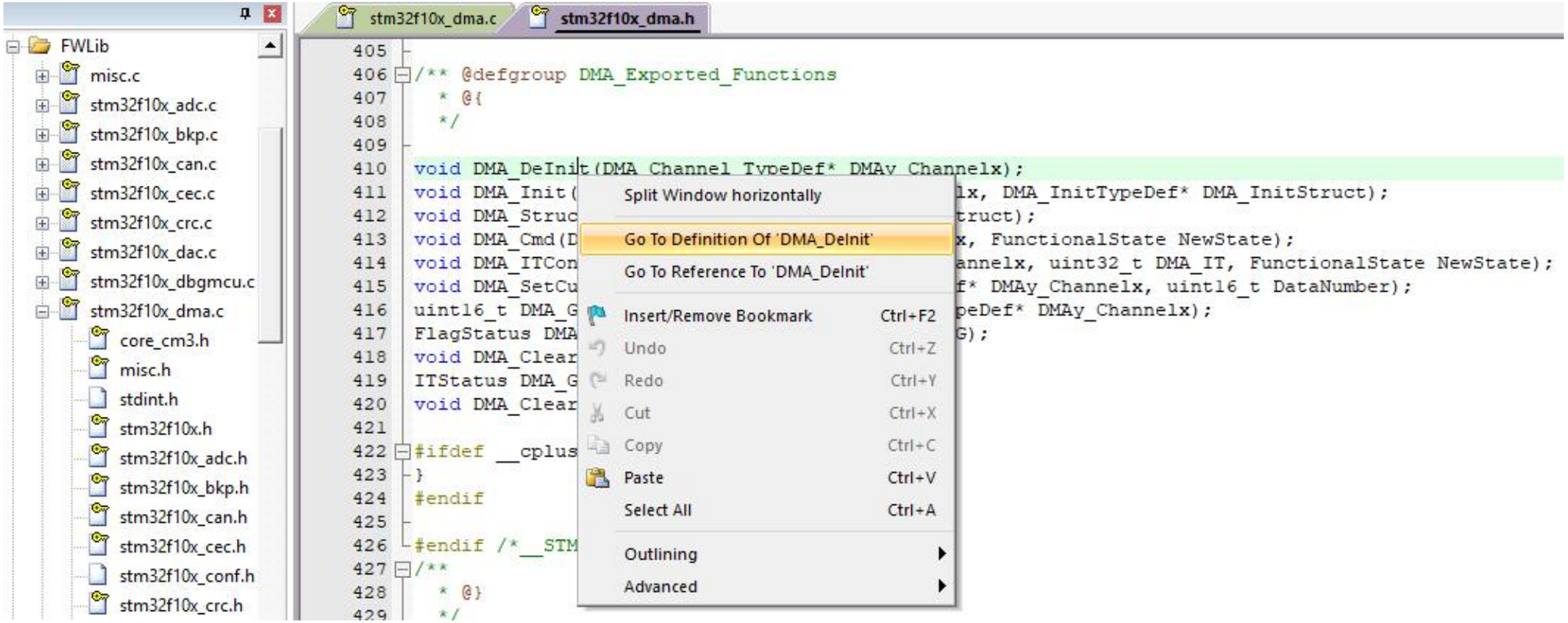
8.3.1 DMA标准外设库接口函数

在stm32f10x_adc.h头文件中声明了ADC所有的库函数



8.3.1 DMA标准外设库接口函数

如果想要查看具体的函数定义，可在keil 工程中将光标放在想要查看的函数上，然后单击右键选择“Go To Definition of… ”，即可跳转到相应函数的函数体



8.3.1 DMA标准外设库接口函数

DMA的初始化函数为：

```
void DMA_Init(DMA_Channel_TypeDef* DMAy_Channelx,
                                   DMA_InitTypeDef*
DMA_Ini
```

参数1： 用于指定DMA的具体通道x

参数2： 根据DMA_InitStruct结构体变量中指定的参数初始化DMA的通道x，DMA_InitStruct是指向DMA_InitTypeDef结构体的指针，包含DMA通道的配置参数

8.3.1 DMA标准外设库接口函数

DMA初始化结构体成员就是配置DMA的相关参数，DMA_InitTypeDef定义在stm32f10x_dma.h文件中

```
typedef struct
{
  uint32_t DMA_PeripheralBaseAddr;  //外设基地址
  uint32_t DMA_MemoryBaseAddr;  //存储器基地址
  uint32_t DMA_DIR;  //传输方向：外设是源还是目的
  uint32_t DMA_BufferSize;  // 缓冲区大小
  uint32_t DMA_PeripheralInc;  //外设地址是否递增
  uint32_t DMA_MemoryInc;  // 存储器地址是否递增
  uint32_t DMA_PeripheralDataSize;  // 外设数据宽度
  uint32_t DMA_MemoryDataSize;  //  存储器数据宽度
  uint32_t DMA_Mode;  // DMA工作模式
  uint32_t DMA_Priority; // DMA优先权
  uint32_t DMA_M2M; // 是否存储器到存储器传输
```

8.3.1 DMA标准外设库接口函数

DMA_InitTypeDef结构体成员及其取值

DMA_InitTypeDef结构体成员	取值范围
DMA_PeripheralBaseAddr(外设基地址)	直接给出外设的地址
DMA_MemoryBaseAddr(存储器基地址)	通常为用户程序定义的主存缓冲区首地址
DMA_DIR(传输方向)	DMA_DIR_PeripheralSRC，外设作为数据传输的来源 DMA_DIR_PeripheralDST，外设作为数据传输的目的地
DMA_BufferSize(缓冲区大小)	设置一次传输数据量的大小，可设为0~65536
DMA_PeripheralInc(外设地址增量)	DMA_PeripheralInc_Enable，外设地址递增允许 DMA_PeripheralInc_Disable，外设地址不变
DMA_MemoryInc(存储器地址增量)	DMA_MemoryInc_Enable，内存地址递增 DMA_MemoryInc_Disable，内存地址不变
DMA_PeripheralDataSize (外设数据宽度)	DMA_PeripheralDataSize_Byte，外设数据宽度为8位 DMA_PeripheralDataSize_HalfWord，外设数据宽度为16位 DMA_PeripheralDataSize_Word，外设数据宽度为32位
DMA_MemoryDataSize (存储器数据宽度)	DMA_MemoryDataSize_Byte，存储器数据宽度为8位 DMA_MemoryDataSize_HalfWord，存储器数据宽度为16位 DMA_MemoryDataSize_Word，存储器数据宽度为32位
DMA_Mode(DMA工作模式)	DMA_Mode_Circular，工作在循环缓冲模式 DMA_Mode_Normal，工作在正常缓冲模式
DMA_Priority(DMA优先权)	DMA_Priority_VeryHigh，最高优先级；DMA_Priority_High，高优先级； DMA_Priority_Medium，中等优先级；DMA_Priority_Low，低优先级
DMA_M2M(存储器到存储器传输)	DMA_M2M_Enable，允许内存到内存的传输； DMA_M2M_Disable，禁止内存到内存的传输

8.3.2 DMA标准外设库的配置过程

库函数的配置过程(以DMA1通道4为例)

初始化DMA通道4参数

允许DMA中断，如果使用中断，则要进行DMA中断配置

```
DMA_ITConfig(DMA1_Channel4,DMA_IT_TC,ENABLE);
```



使能DMA时钟

DMA挂接在AHB，所以使用RCC_AHBPeriphClockCmd()函数使能DMA时钟：

```
RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,ENABLE);
```

使能DMA1通道4，启动传输

```
DMA_Cmd (DMA1_Channel4, ENABLE);
```

8.3.2 DMA标准外设库的配置过程

```
DMA_InitTypeDef DMA_InitStructure;
   /*设置DMA源：串口数据寄存器地址*/
DMA_InitStructure.DMA_PeripheralBaseAddr
=(uint32_t)(&(USART1->DR));
   /*内存地址，要传输的变量的指针*/
DMA_InitStructure.DMA_MemoryBaseAddr =
(uint32_t)SendBuff;
   /*方向：从内存到外设*/
DMA_InitStructure.DMA_DIR =
DMA_DIR_PeripheralDST;
   /*传输大小DMA_BufferSize=SENDBUFF_SIZE*/
DMA_InitStructure.DMA_BufferSize =
SENDBUFF_SIZE;
   /*外设地址设置为不增*/
DMA_InitStructure.DMA_PeripheralInc =
DMA_PeripheralInc_Disable;
   /*内存地址设置为自增*/
DMA_InitStructure.DMA_MemoryInc =
DMA_MemoryInc_Enable;
```

初始化DMA通道4参数

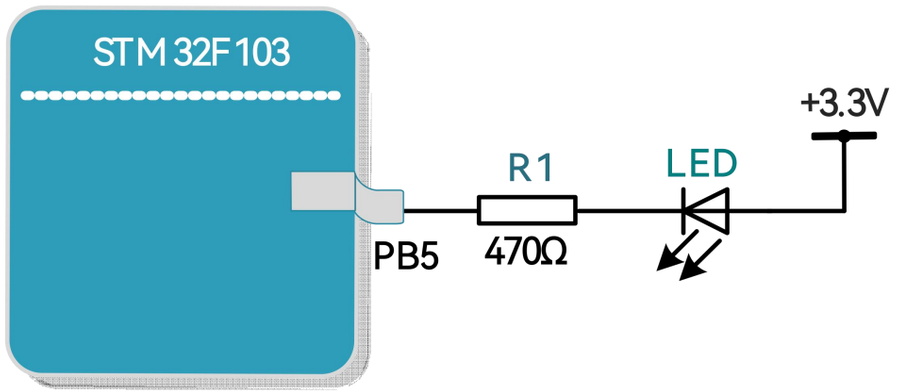
```
    /*设置外设数据单位*
DMA_InitStructure.DMA_PeripheralDataSize
= DMA_PeripheralDataSize_Byte;
    /*设置内存数据单位8bit*/
DMA_InitStructure.DMA_MemoryDataSize =
DMA_MemoryDataSize_Byte;
   /*设置DMA模式为不断循环模式*/
DMA_InitStructure.DMA_Mode =
DMA_Mode_Circular;
   /*设置DMA的优先级：中*/
DMA_InitStructure.DMA_Priority =
DMA_Priority_Medium;
   /*禁止内存到内存的传输*/
DMA_InitStructure.DMA_M2M =
DMA_M2M_Disable;
DMA_Init(DMA1_Channel4,
&DMA_InitStructure);
```

8.3.3 DMA标准外设库应用实例

功能

通过DMA方式将数据发送到USART1接口，并通过PC端串口调试助手显示传输结果。为证明DMA方式不需要CPU直接控制，在主程序中通过循环点亮LED灯进行显示。

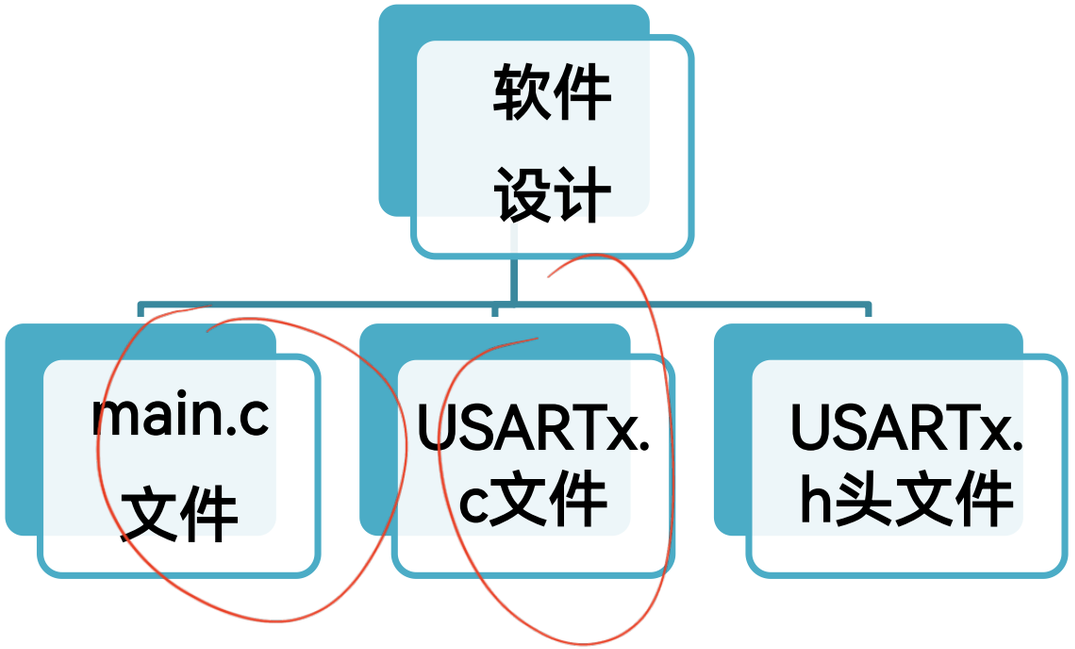
硬件设计



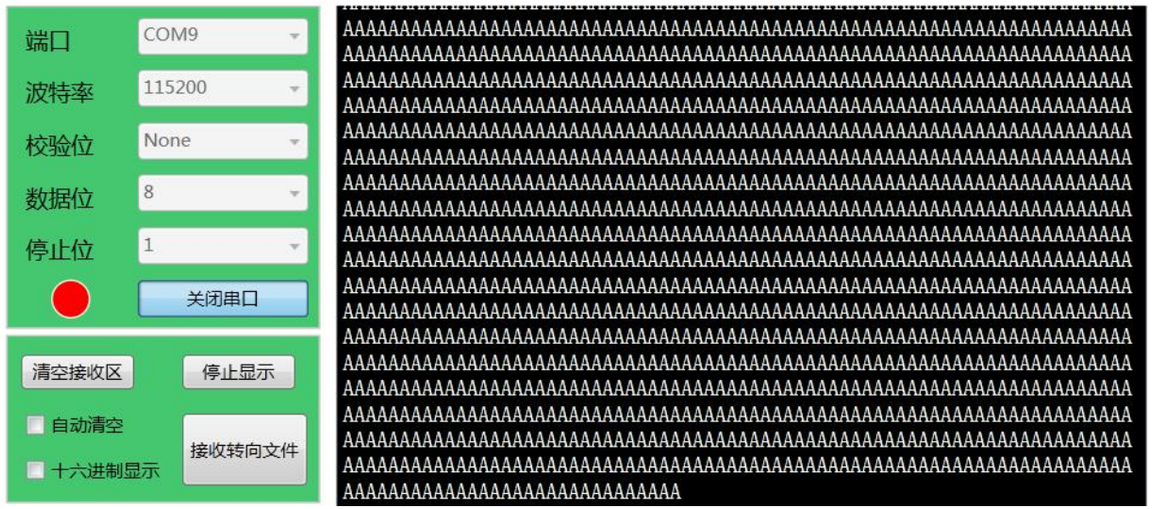
本实例DMA不需要硬件，主程序所用到的LED灯的硬件设计如图。

8.3.3 DMA标准外设库应用实例

注释，选10处



编译无错误后下载到开发板，通过串口调试助手可以观察到实验现象：主程序LED灯一直循环闪烁，数据通过DMA方式不停地发送A到串口。



8.3.3 DMA标准外设库应用实例

```
#include "stm32f10x.h"
#include "led.h"
#include "USARTx.h"

extern uint8_t SendBuff[SENDBUFF_SIZE]; //
SENDBUFF_SIZE变量定义在USARTx.h头文件中

//不精确延时
static void Delay(uint32_t time)
{
  uint32_t i,j;
  for(i=0;i<time;++i)
  {
    for(j=0;j<10000;++j)
    {
      //空循环体，什么都不做
    }
  }
}
```

main.c文件

```
int main(void)
{
    uint16_t i;

    LED_Init(); //LED灯初始化
    USARTx_Init_Config();//USART1初始化
    USARTx_DMA_Config(); //DMA初始化
    printf("使用DMA方式传输串口数据\n");
       /*输入要发送的数据，作为DMA传输的数据源*/
    for(i=0;i<SENDBUFF_SIZE;i++)
    {
       SendBuff[i] =  'A';
    }
         /* USART1 向DMA发出传输请求*/
    USART_DMACmd(USART1, USART_DMAReq_Tx,
ENABLE);
    while(1)
    {
         GPIO_SetBits(GPIOB,GPIO_Pin_5); //熄灭LED灯
         Delay(1000);
         GPIO_ResetBits(GPIOB,GPIO_Pin_5); //点亮LED灯
          Delay(1000);
    }
}
```

8.3.3 DMA标准外设库应用实例

USARTx.c文件

```
#include "USARTx.h"
uint8_t SendBuff[SENDBUFF_SIZE];

void USARTx_Init_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
                    USART_InitTypeDef USART_InitStructure;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RC
C_APB2Periph_GPIOA, ENABLE);
                    USART_DeInit(USART1);
        //USART1_TX   PA.9
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9; //PA.9
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;

    GPIO_Init(GPIOA, &GPIO_InitStructure);
        //USART1_RX  PA.10
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    //USART1 参数配置
USART_InitStructure.USART_BaudRate = 115200;
USART_InitStructure.USART_WordLength =
USART_WordLength_8b;
USART_InitStructure.USART_StopBits =
USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode = USART_Mode_Rx |
USART_Mode_Tx;
USART_Init(USART1, &USART_InitStructure);
USART_Cmd(USART1, ENABLE);
    /* 清除发送完成标志*/
USART_ClearFlag(USART1,
USART_FLAG_TC|USART_FLAG_TXE|USART_FLAG_RXN
E);
}
```

8.3.3 DMA标准外设库应用实例

```
void USARTx_DMA_Config(void)
{
  DMA_InitTypeDef DMA_InitStructure;
    /*开启DMA时钟*/
  RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,
ENABLE);
    /*设置DMA源：串口数据寄存器地址*/
  DMA_InitStructure.DMA_PeripheralBaseAddr
=(uint32_t)(&(USART1->DR));
    /*内存地址，要传输的变量的指针*/
  DMA_InitStructure.DMA_MemoryBaseAddr =
(uint32_t)SendBuff;
    /*方向：从内存到外设*
  DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST;

    /*传输大小DMA_BufferSize=SENDBUFF_SIZE*/
  DMA_InitStructure.DMA_BufferSize = SENDBUFF_SIZE;
    /*外设地址设置为不增*
  DMA_InitStructure.DMA_PeripheralInc =
DMA_PeripheralInc_Disable;
```

USARTx.c文件

```
 /*内存地址设置为自增*/
  DMA_InitStructure.DMA_MemoryInc =
DMA_MemoryInc_Enable;
    /*设置外设数据单位*
  DMA_InitStructure.DMA_PeripheralDataSize =
DMA_PeripheralDataSize_Byte;
    /*设置内存数据单位8bit*/
  DMA_InitStructure.DMA_MemoryDataSize =
DMA_MemoryDataSize_Byte;
    /*设置DMA模式为不断循环模式*/
  DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;

    /*设置DMA的优先级：中*
  DMA_InitStructure.DMA_Priority =
DMA_Priority_Medium;
    /*禁止内存到内存的传输*/
  DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
    /*配置DMA1的通道4*/
  DMA_Init(DMA1_Channel4, &DMA_InitStructure);
  /*使能DMA*/
  DMA_Cmd (DMA1_Channel4,ENABLE);

    /*设置DMA发送完成后产生中断 */
  DMA_ITConfig(DMA1_Channel4,DMA_IT_TC,ENABLE);
}
```

8.3.3 DMA标准外设库应用实例

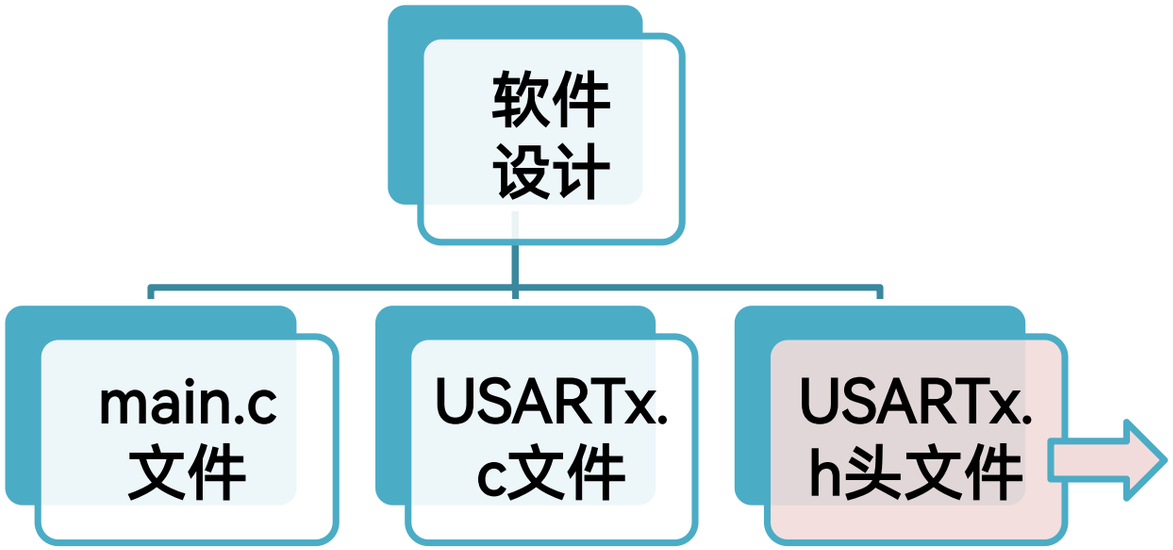
USARTx.c文件

```
    // 重定向C库函数printf到USART1
int fputc(int ch, FILE *f)
{
        /* 发送一个字节数据到串口USART1*/
    USART_SendData(USART1, (uint8_t) ch);
        /* 等待串口数据发送完毕*/
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);

      return (ch);
}

      //重定向c库函数scanf到USART1
int fgetc(FILE *f)
{
        /* 等待串口输入数据*/
    while (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET);
      return (int)USART_ReceiveData(USART1);
}
```

8.3.3 DMA标准外设库应用实例



USARTx.h文件

```
#ifndef __USARTx_H
#define __USARTx_H

#include "stm32f10x.h"
#include <stdio.h>

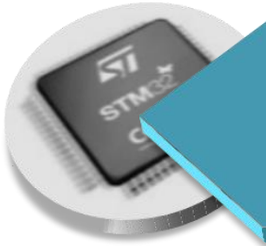
#define  SENDBUFF_SIZE    100

void USARTx_Init_Config(void);
void USARTx_DMA_Config(void);

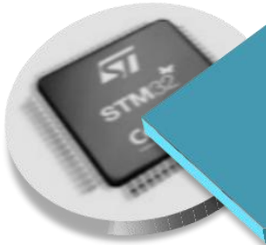
int fputc(int ch, FILE *f);
int fgetc(FILE *f);

#endif
```

8.4 DMA模块的HAL库接口函数及应用



8.4.1 DMA的HAL库接口函数



8.4.2 HAL库DMA应用实例

8.4.1 DMA的HAL库接口函数

DMA模块的HAL库相关的接口函数定义在stm32f1xx_hal_dma.c源文件中，在stm32f1xx_hal_dma.h头文件中可以查看相关函数的声明以及结构体定义。

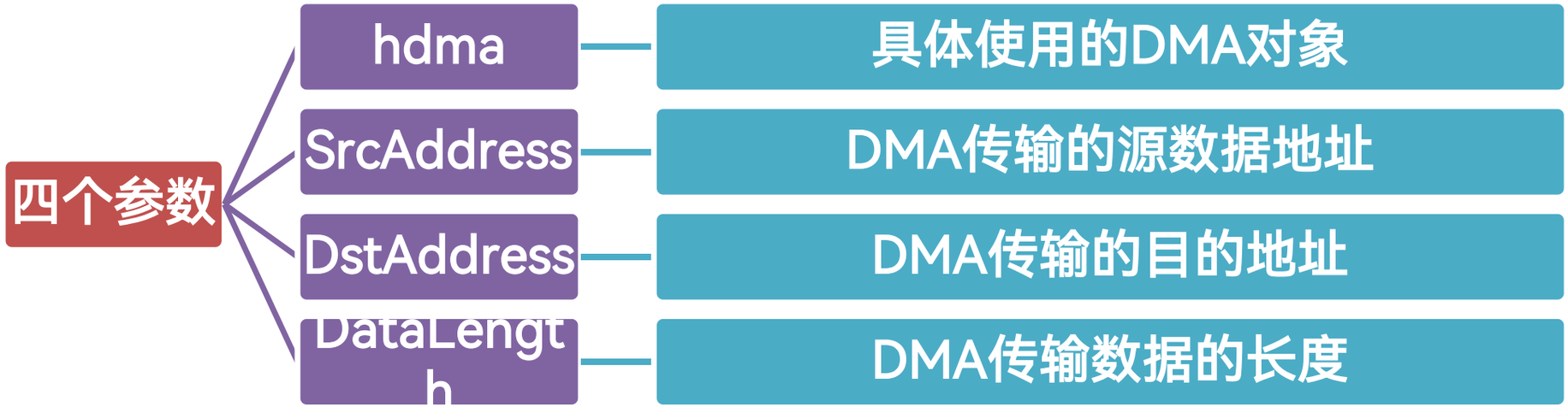
STM32的HAL库常用DMA接口函数

类型	函数及功能描述
初始化及复位函数	HAL_DMA_Init(DMA_HandleTypeDef *hdma); 功能描述：DMA初始化函数
	HAL_DMA_DeInit (DMA_HandleTypeDef *hdma); 功能描述：DMA复位函数
引脚功能操作函数	HAL_DMA_Start (DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：启动DMA传输
	HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：DMA中断开始
	void HAL_DMA_IRQHandler(DMA_HandleTypeDef *hdma); 功能描述：DMA中断处理函数
外设状态函数	HAL_DMA_GetState(DMA_HandleTypeDef *hdma); 功能描述：获取DMA状态函数
	HAL_DMA_GetError(DMA_HandleTypeDef *hdma); 功能描述：获取DMA错误函数

8.4.1 DMA的HAL库接口函数

DMA启动传输函数
HAL_DMA_Start()

```
HAL_DMA_Start(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t
DataLength);
```



使用范例：

```
HAL_DMA_Start(huart->hdmatx, (u32)pData, (uint32_t)&huart->Instance->DR, 100);
```

//开启DMA的串口传输

8.4.1 DMA的HAL库接口函数

DMA串口发送数据传输函数 AL_UART_Transmit_DMA()

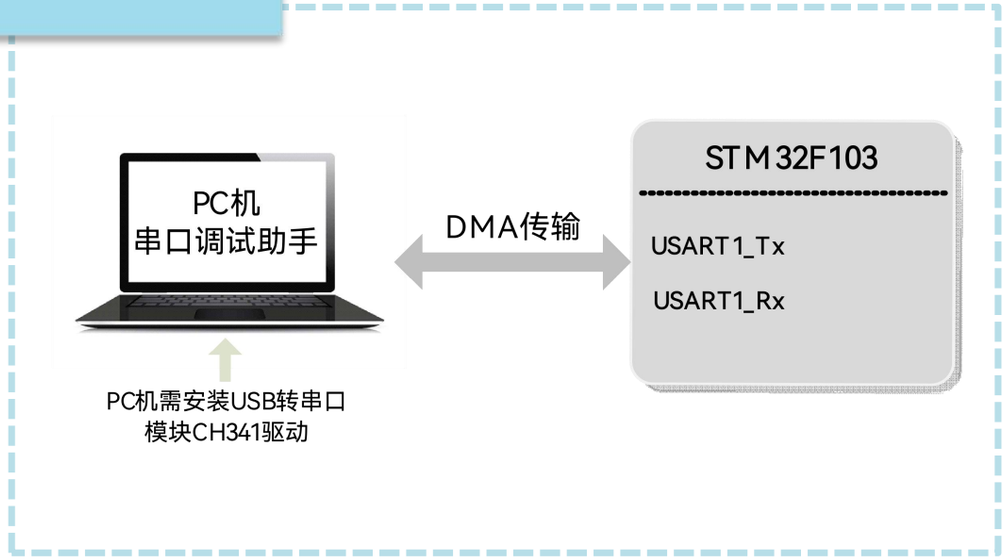
```
HAL_StatusTypeDef
HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size)
{
  uint32_t *tmp;
  if (huart->gState == HAL_UART_STATE_READY)
  {
    if ((pData == NULL) || (Size == 0U))
    {
      return HAL_ERROR;
    }
     /*huart配置，上锁 */
    __HAL_LOCK(huart);
    huart->pTxBuffPtr = pData;
    huart->TxXferSize = Size;
    huart->TxXferCount = Size;
    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->gState = HAL_UART_STATE_BUSY_TX;
     /*设置UART的DMA传输完成时调用的回调函数*/
    huart->hdmatx->XferCpltCallback =
UART_DMATransmitCplt;
     /*设置UART的DMA传输完成一半时的回调函数 */
    huart->hdmatx->XferHalfCpltCallback =
UART_DMATxHalfCplt;
     /*设置DMA传输错误的回调函数 */
    huart->hdmatx->XferErrorCallback =
UART_DMAError;
     /*置DMA传输中止的回调函数*/
    huart->hdmatx->XferAbortCallback = NULL;
     /*设置存储器到外设的地址，开启中断，并使能UART
的DMA传输的DMA通道*/
    tmp = (uint32_t *)&pData;
    HAL_DMA_Start_IT(huart->hdmatx, *(uint32_t *)tmp,
(uint32_t)&huart->Instance->DR, Size);
     /*清除串口TC中断标志位 */
    __HAL_UART_CLEAR_FLAG(huart, UART_FLAG_TC);
     /* huart解锁 */
    __HAL_UNLOCK(huart);
     /* 通过配置UART的CR3寄存器，使能DMA发送*/
    SET_BIT(huart->Instance->CR3, USART_CR3_DMAT);
    return HAL_OK;
  }
  else
  {   return HAL_BUSY;   }
}
```

8.4.2 HAL库DMA应用实例

功能

基于HAL库采用DMA方式实现USART串口收发数据，用于需要大批量数据的通信与信息交换的场合。

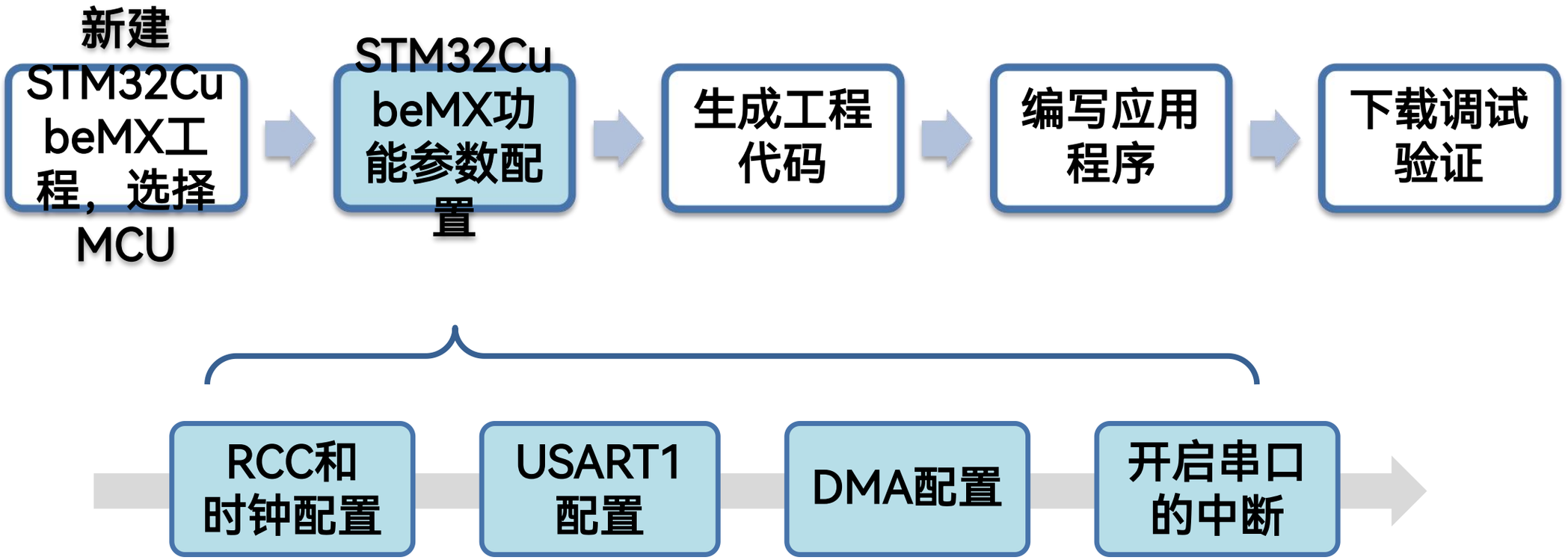
硬件设计



通过STM32F103目标板上的串口1（USART1）采用DMA方式实现与上位机PC的数据传输，可在PC机的串口助手查看测试结果。

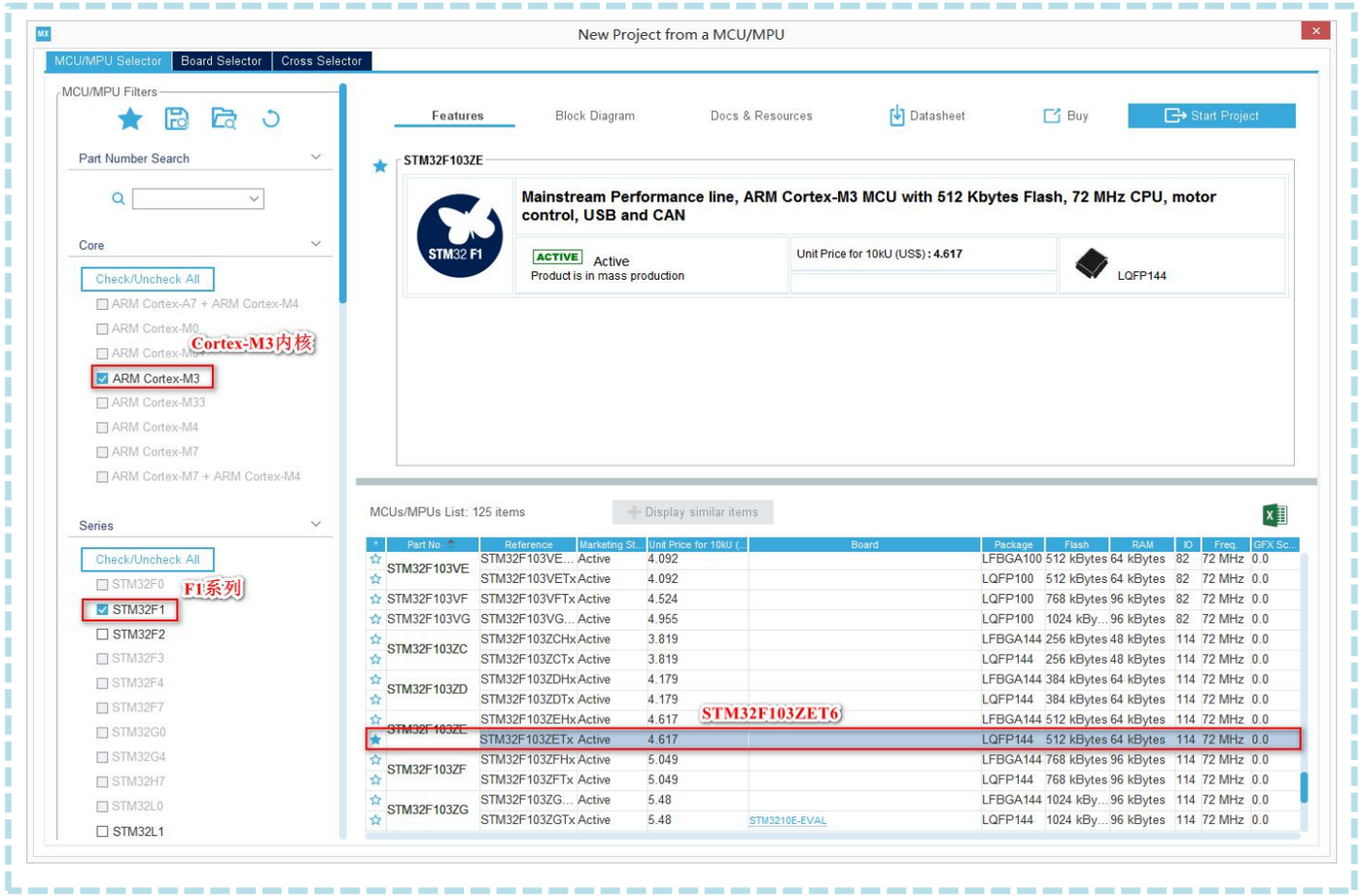
8.4.2 HAL库DMA应用实例

软件设计流程



软件设计——新建STM32CubeMX工程，选择MCU

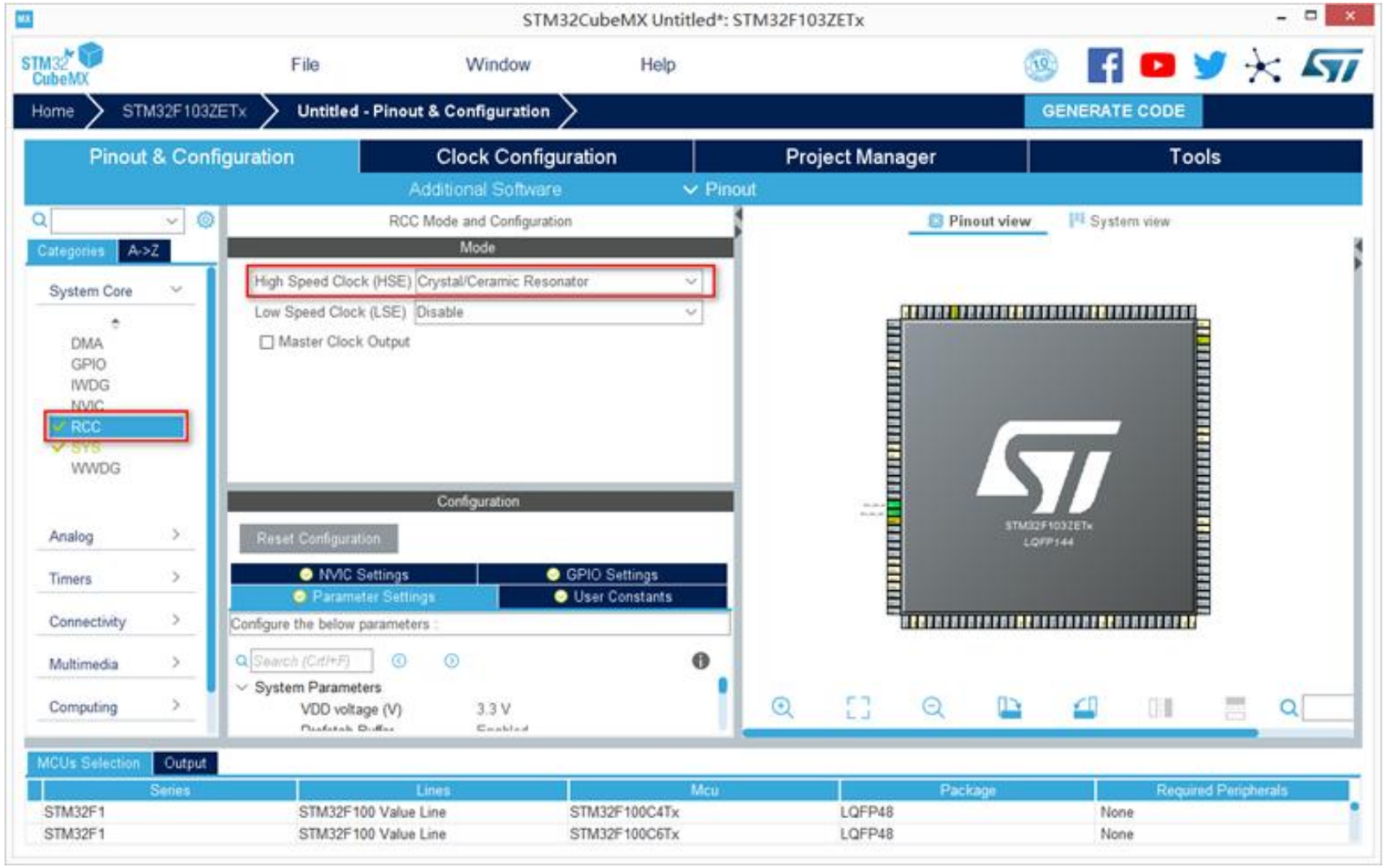
新建一个STM32CubeMx工程，选择选择MCU，这里选择STM32F103ZETx芯片，读者可根据自己的目标板选择相应的芯片



软件设计——STM32CubeMX功能参数配置

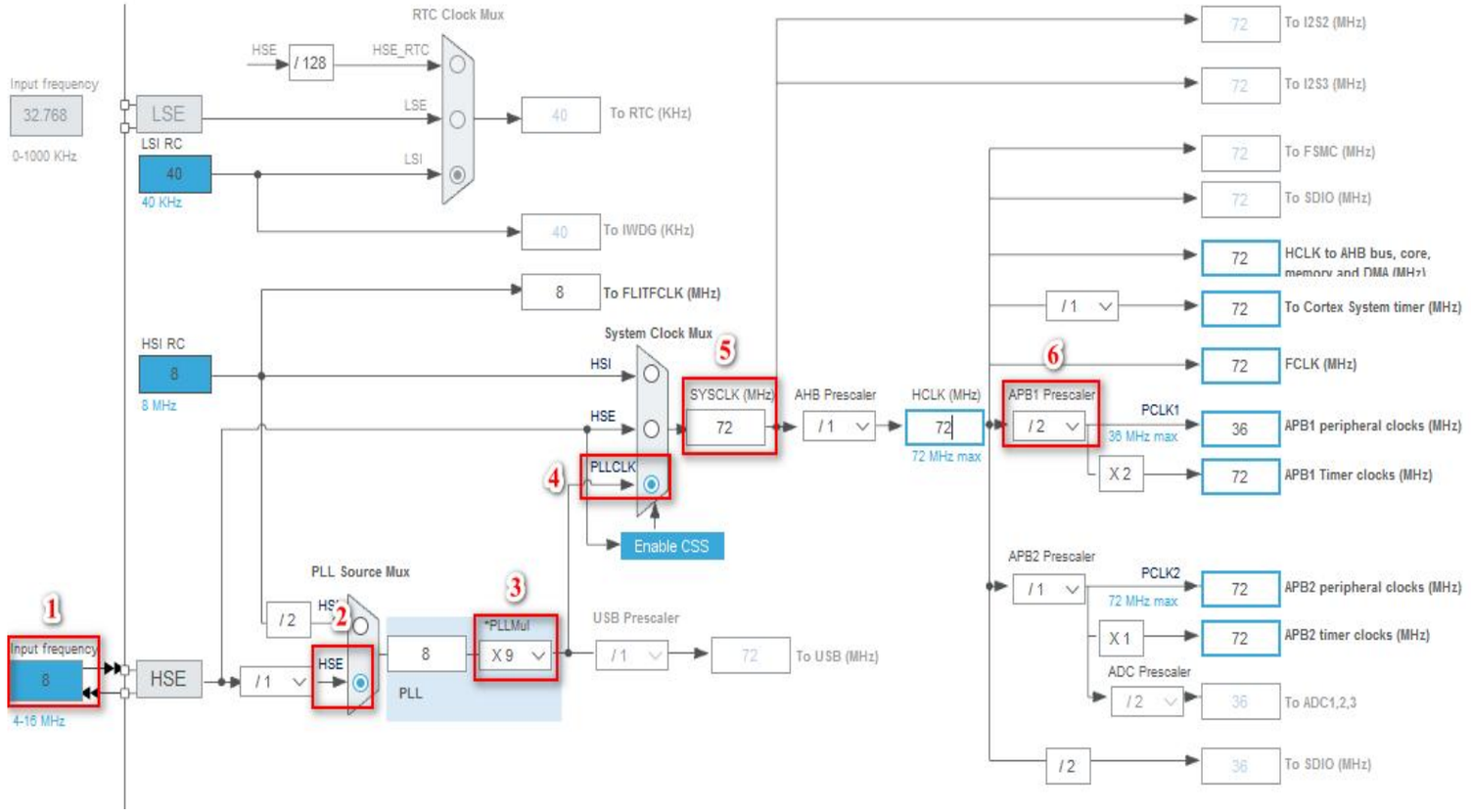
RCC配置

HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器



软件设计——STM32CubeMX功能参数配置

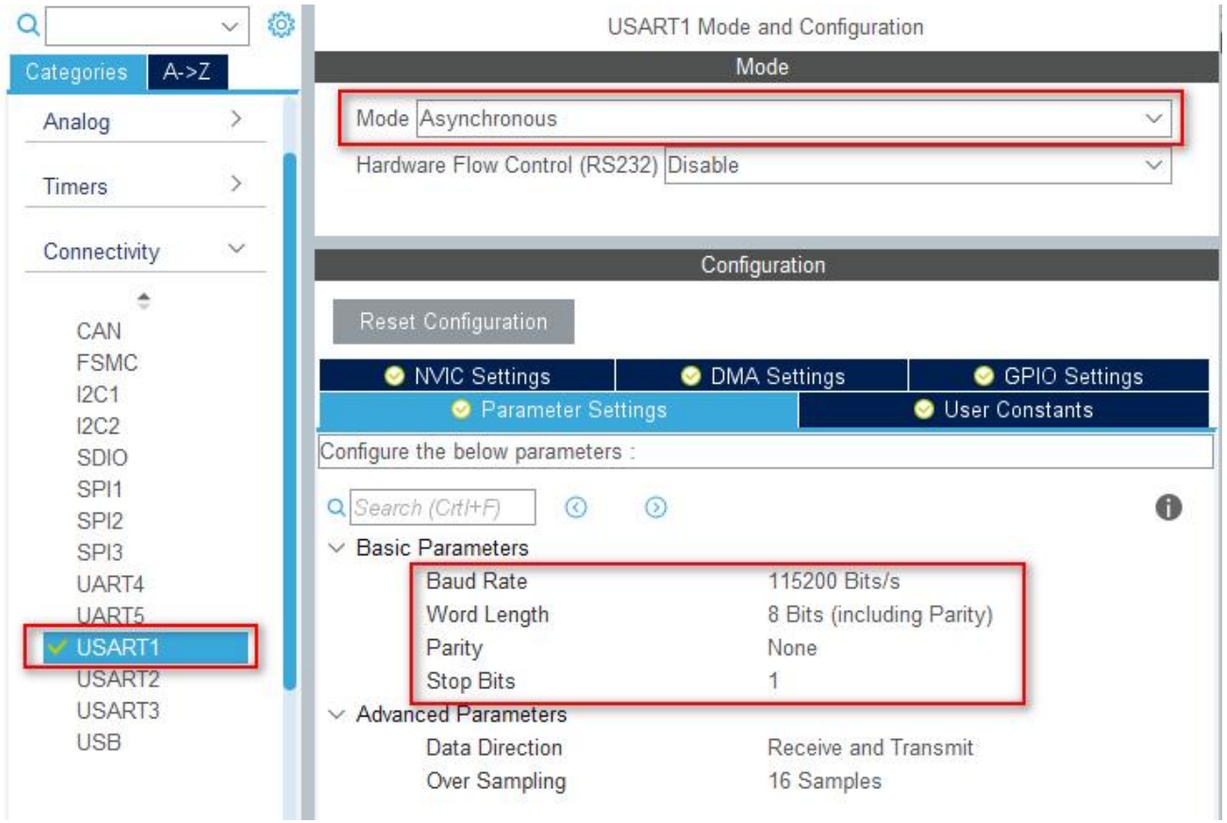
时钟配置



通过图形化方式直观地对系统时钟进行配置，系统时钟采用外部高速时钟，配置STM32F103系列芯片最大时钟为72MHz，配置APB2为72MHz，配置APB1为36MHz。

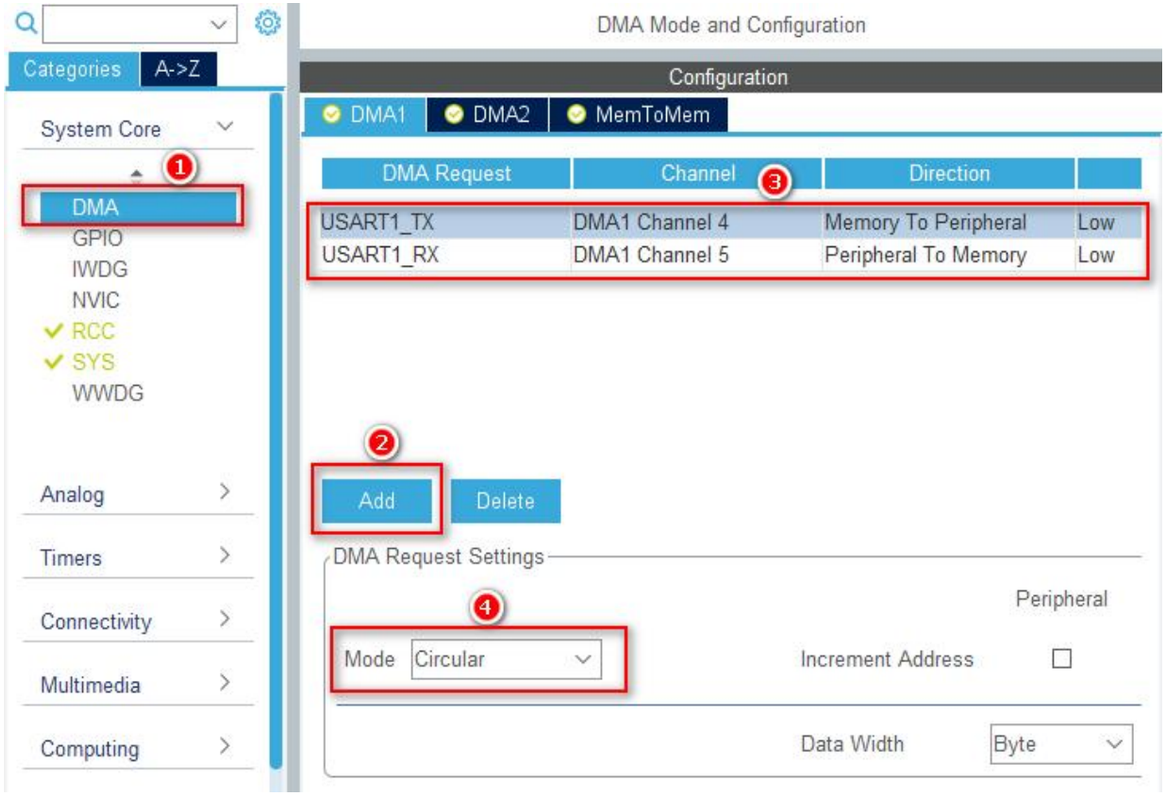
软件设计——STM32CubeMX功能参数配置

串口USART1配置



- 在“Connectivity”中选择“USART1”；
- 设置“MODE”为“Asynchronous”；
- 设置USART1的相关参数配置为默认的：115200、8、None和1

软件设计——STM32CubeMX功能参数配置

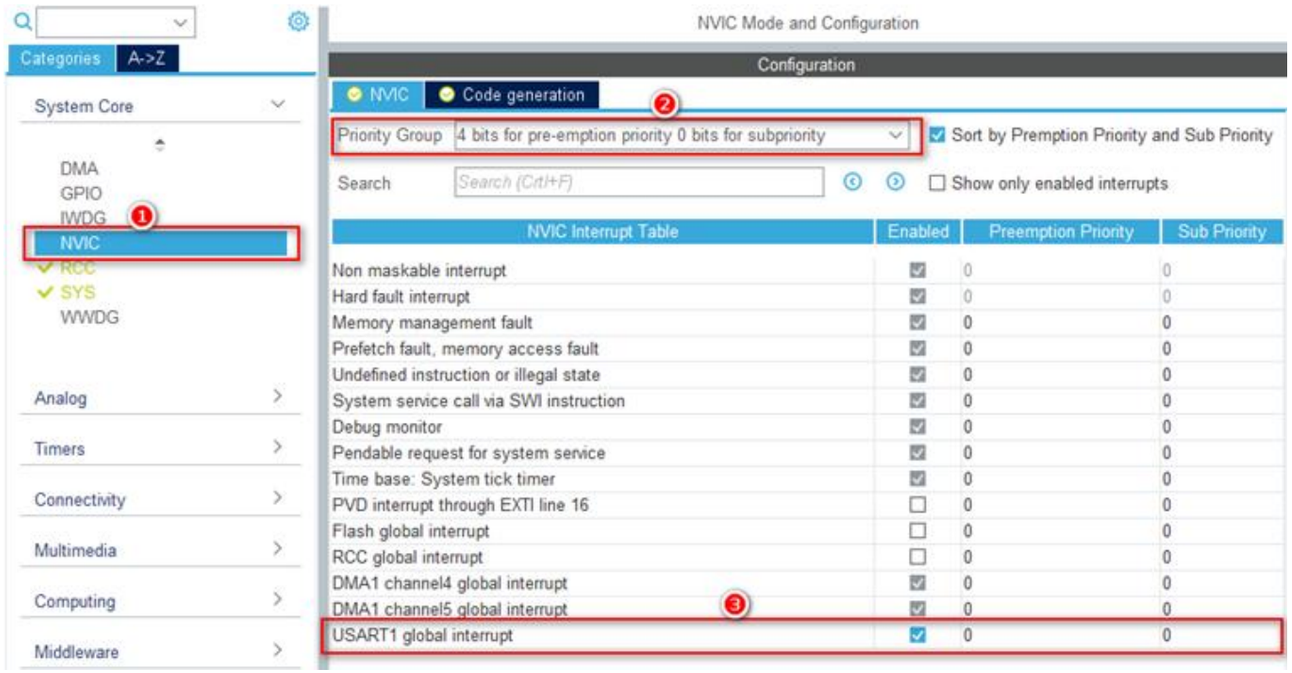


DMA配置

- 在“System Core”中单击“DMA”
- 在右侧的配置页面“Configuration”中，单击“Add”，添加“USART1_TX”；
- “Mode”选择为“Circular”；
- “Channel”和“Direction”会自动选择；
- “Data Width”用默认的“Byte”；
- 按同样的操作方式添加“USART1_RX”

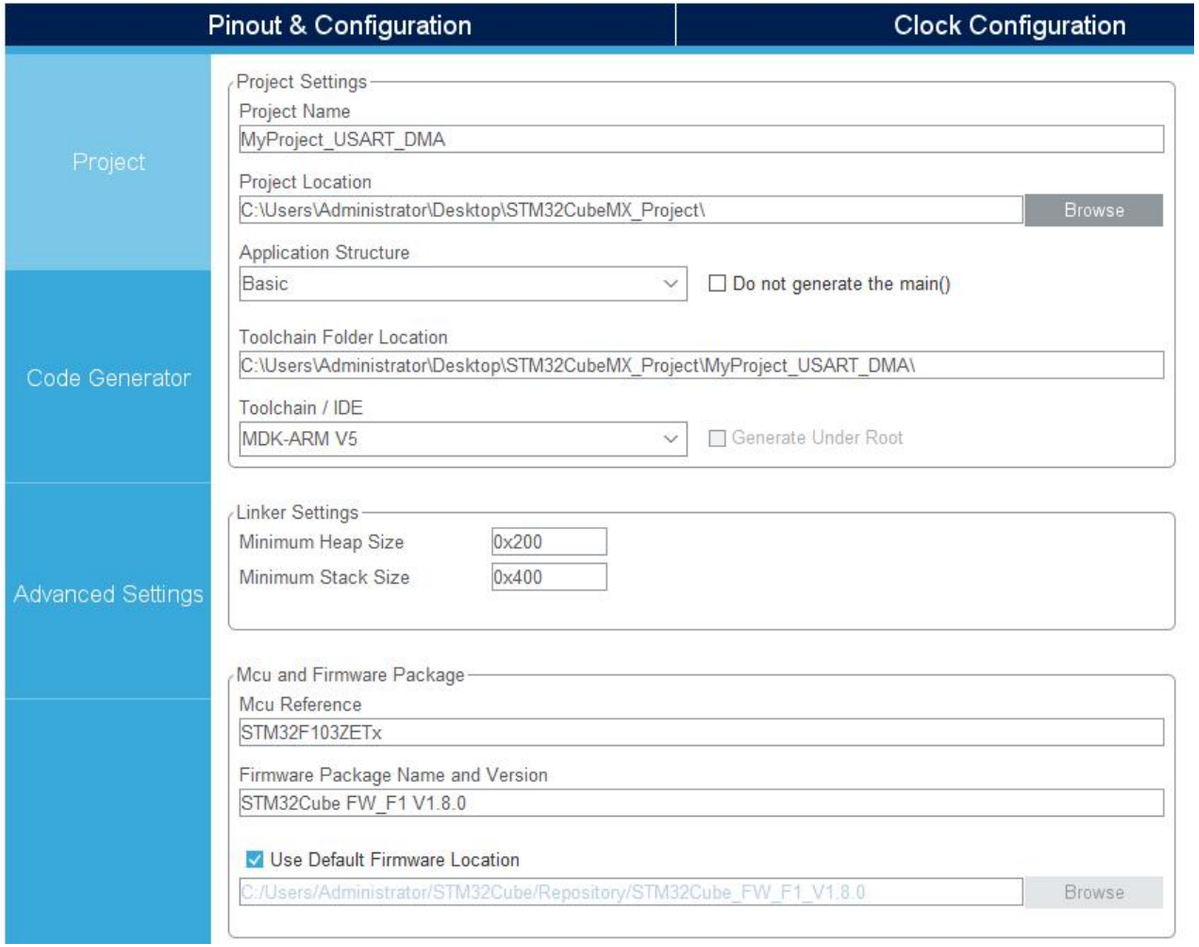
软件设计——STM32CubeMX功能参数配置

开启串口USART1的中断



- 在“System Core”中点击“NVIC”，在右侧的配置页面“Configuration”中，可以设置“Priority Group”优先级分组，默认为“4 bits for pre-emption priority 0 bits for subpriority”；
- 勾选“USART1 global interrupt”，“Priority Group”优先级分组可根据具体需要选择合适的分组。

软件设计——生成工程代码

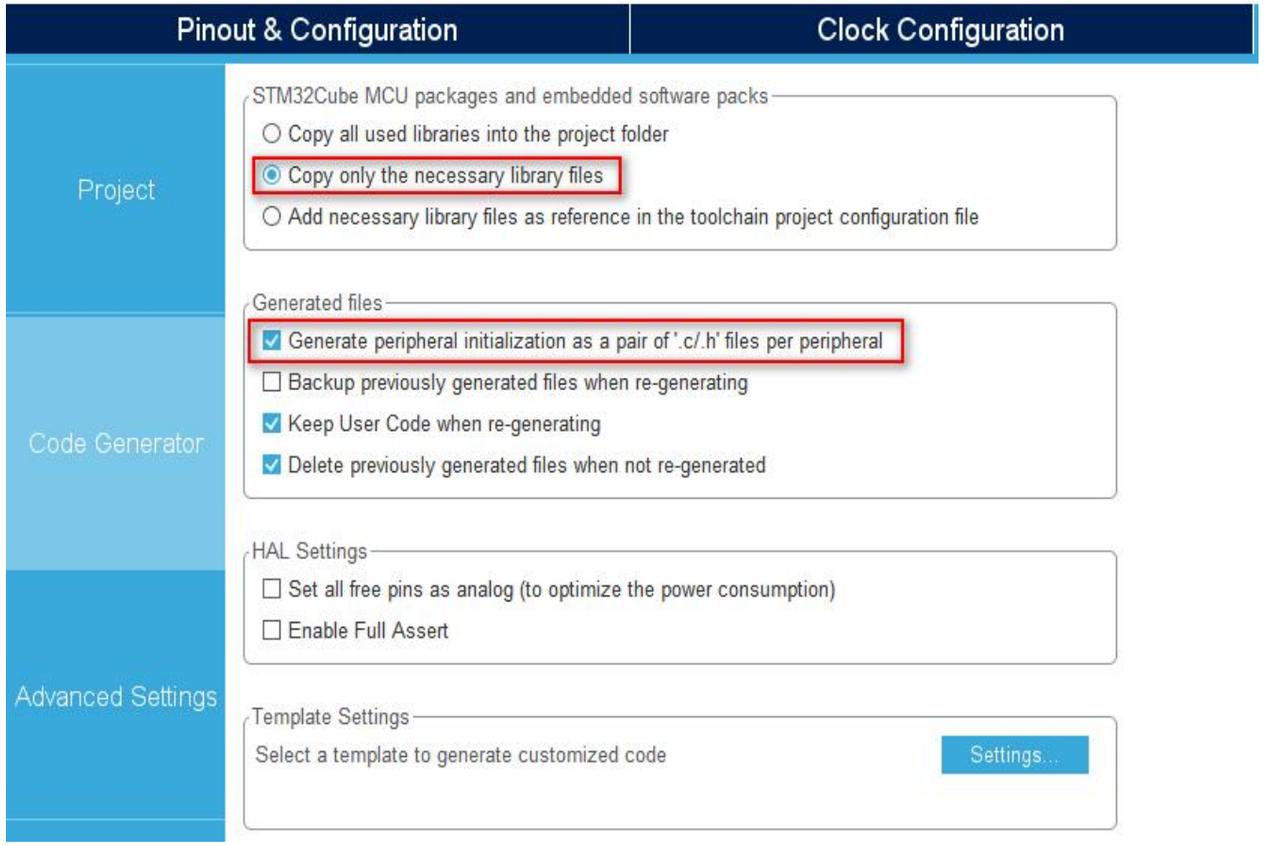


配置keil工程名称和存放位置

配置工程名称、工程保存位置、选择“MDK-ARM V5”编译器等



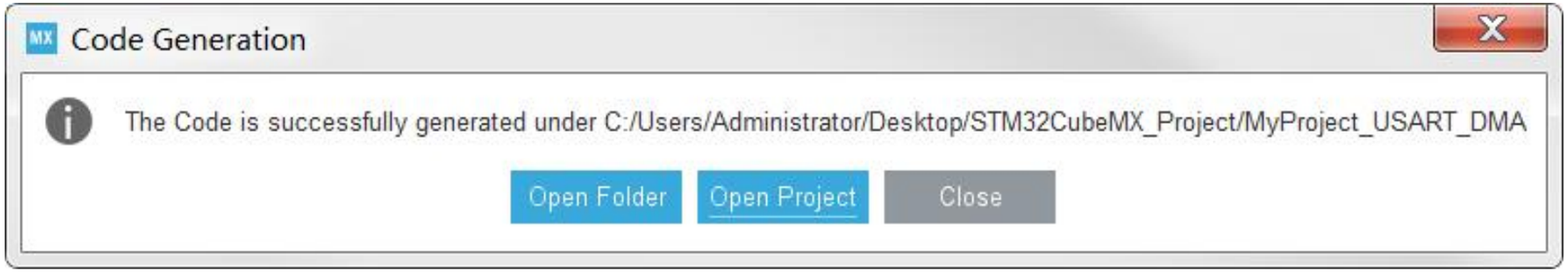
软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计——编写应用程序

在main.c文件中添加代码：

- 先定义两个数组DMA_RxBuffer和DMA_TxBuffer，用于接收数据和发送数据
- 将此代码放在main.c文件中的/* USER CODE BEGIN PV */和/* USER CODE END PV */之间。

```
/* Private variables ---------------------------------------*/
/* USER CODE BEGIN PV */
/* Private variables ---------------------------------------*/
uint8_t DMA_RxBuffer[10];
uint8_t DMA_TxBuffer[] = " This is the sending code with DMA  \r\n ";
/* USER CODE END PV */
```

软件设计——编写应用程序

在int main(void)函数中的/* USER CODE BEGIN 2 */和 /* USER CODE END 2 */之间添加代码：

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_DMA(&huart1,DMA_RxBuffer,10);// 开启DMA接收数据
HAL_UART_Transmit_DMA(&huart1,(uint8_t *)DMA_TxBuffer,sizeof(DMA_TxBuffer));
/* USER CODE END 2 */
```

软件设计——编写应用程序

在/* USER CODE BEGIN 4 */和/* USER CODE END 4 */之间添加如下代码：

```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  HAL_UART_Transmit_DMA(&huart1,DMA_RxBuffer,10);
  HAL_UART_Receive_DMA(&huart1,DMA_RxBuffer,10);  //重新打开DMA接收数据
}
/* USER CODE END 4 */
```

本章小结

8.1 DMA基础理论知识

8.2 STM32的DMA模块

- DMA内部结构
- DMA优先权
- DMA中断请求

8.3 DMA模块的标准外设库接口函数及应用

- 接口函数
- 配置过程
- 应用实例

8.4 DMA模块的HAL库接口函数及应用

- 接口函数
- 应用实例



The End!

